



LINUX FUNDAMENTALS

Parsa M.

Discord: corelarc

Thursday, October 1, 2026 / Hall 929 / 6–8 pm

Table of Contents

Topic	Focus
CLI vs GUI	Terminology, command-line interface, command structure
Remote Shell - SSH	Connecting to campus linux servers via ssh
Linux File System	File system hierarchy, files and directories
File System Navigation	Navigating the filesystem, finding or creating a file/directory
Essential Commands	Linux must know commands
Vim	Vim, Vim motions and vimtutor

Terminology

Term	What it means today
Command-Line Interface	A text based interface in which one interacts with the computer via commands
Terminal	Program that provides (emulates) cli in a graphical environment
Shell	The program that interprets commands (E.g Bash, Zsh, Fish)
SSH	Secure SHell - a communication protocol
Directory	Equivalent to a Windows' "Folder"

Your laptop can run Windows or macOS. The commands run on the Linux server.

A terminal on your laptop

Operating System

Open this

Windows

Windows Terminal or PowerShell

macOS

Terminal

Linux

Your terminal application

SSH into Concordia

LOCAL TERMINAL

```
ssh ENCS_USERNAME@login.encs.concordia.ca
```

ENCS_USERNAME

Replace this with your actual ENCS (netname) username.

Password

Use your ENCS password.
Typing shows no characters.

First connection (type yes)

Verify the host fingerprint through trusted university information.

Type yes and then enter

Who am I? Where am I?

Getting the lay of the land

Command \$

What does it do?

\$ whoami

Prints your username on the terminal

\$ pwd

Shows the absolute path to the current directory

\$ ls

Lists all the files and directories on the current directory

\$ cd

Change directory

\$ mkdir

Make a new directory

Structure of a Linux Command

REMOTE BASH

```
$ ls -a
```

ls

Command name

```
$ ls -l
```

-a -l -h

Options that change its behaviour

```
$ ls -al
```

```
$ ls -al projects/
```

projects

Argument: the directory to inspect

Commands and paths are case-sensitive. The \$ prompt is not part of a command.

Where to Find a Command's Syntax?

To learn more about a command, simply ask ma man: man (manual)

```
$ man {command}
```

```
$ man ls
```

```
$ ls --help
```

```
$ git -h
```

```
$ man -k {keyword}
```

Usually for a quick overview of a command's structure/syntax we use the --help flag over its manual.

How to Navigate a Man Page?

Key	Purpose
Space or f	Page forward
b	Page backward
d / u	half page forward/backward
g	Jump to the first line
G	Jump to last line
q	quit

How to Search a Man Page?

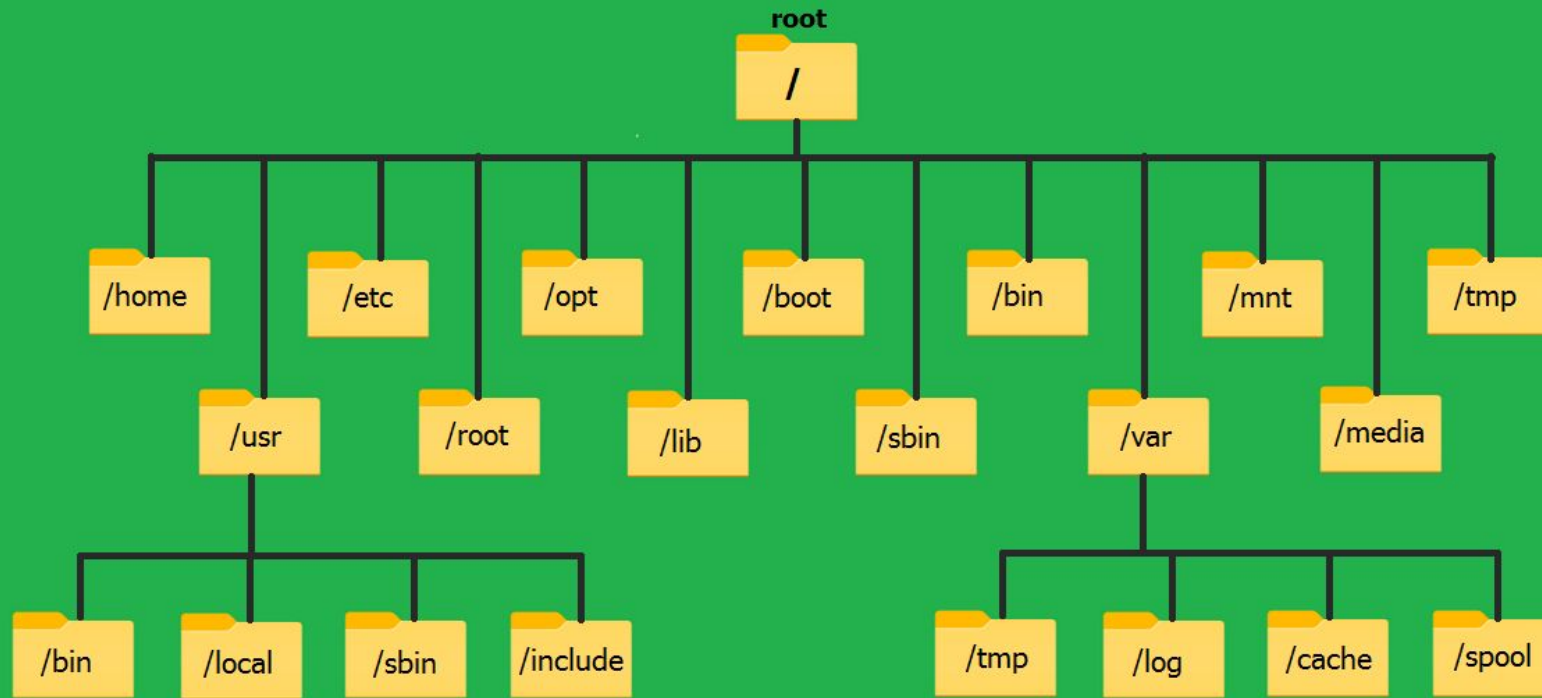
Key	Purpose
/ {pattern}	Search any occurrence of a given {pattern} from first occurrence
? {pattern}	Reverse searches for a given {pattern} from last occurrence
n	Next match
N	Previous match
G	Jump to last line
q	quit

The Filesystem

Command	Function
<code>\$ tree</code>	Visualizes the filesystem hierarchy
<code>\$ tree -d</code>	Directories only
<code>\$ tree -a</code>	Include hidden files and directories (dotfiles)
<code>\$ tree -L 2</code>	Set a depth of 2 levels

/ is the filesystem root. Linux filesystem is a tree and has only one root: /

The Filesystem hierarchy



File System Hierarchy(FHS) of Linux

Absolute and relative paths

Path	Meaning
<code>/home/{c}/{net_name}</code>	Absolute: starts at /
<code>code/main.c</code>	Relative: starts at the current directory
<code>.</code>	The current directory
<code>..</code>	The parent directory
<code>~</code>	A shortcut for the current user's home directory
<code>~/hexploit-linux</code>	Your home, then the practice folder

A leading slash changes where a path starts. A leading dot can make a filename hidden.

Navigation that saves typing

REMOTE BASH

pwd

cd Downloads

cd ..

cd ~

cd -

cd ~/code

cd ..

One directory upward

cd or **cd** ~

Back to your home

cd -

Back to the previous directory

pwd

Check your location before changing files

Populating/Purging the FileSystem

Command	What it does
<code>\$ touch {filename}</code>	Creates a file
<code>\$ mkdir {directory name}</code>	Creates a directory
<code>\$ rm {filename}</code>	Permanently deletes a file
<code>\$ rmdir {directory name}</code>	Deletes an empty directory
<code>\$ rm -rf {directory}</code>	Recursively deletes all files in a directory

Recursive deletion: `rm -r DIRECTORY` removes a directory and its contents. Use only on disposable data.

Copy, Move and Rename

Command

Function

```
$ cp {source}  
{destination}
```

Copies a file

```
$ cp -r {source}  
{destination}
```

Recursively copies a file

```
$ mv {source}  
{destination}
```

Moves a file in a new location

```
$ mv {oldname}  
{newname}
```

Renames a file

Creating, copying, and moving

Command

```
$ mkdir -p scratch/archive
```

```
$ touch scratch/draft.txt
```

```
$ cp -i docs/notes.txt scratch/notes.txt
```

```
$ mv -i scratch/notes.txt scratch/archive/
```

```
$ cp -r docs scratch/docs-copy
```

mkdir -p

Create missing parent directories

touch

Create an empty file, or update timestamps

cp / mv

Copy / move or rename

-i

Ask before overwriting an existing file

An existing destination directory receives the source under its original name.

Reading files without an editor

Command	Use
<code>\$ cat docs/notes.txt</code>	Print a small file
<code>\$ less logs/app.log</code>	Page through a longer file
<code>\$ head -n 5 logs/app.log</code>	Show the first five lines
<code>\$ tail -n 5 logs/app.log</code>	Show the last five lines

In less: Space next page, b previous page, /{pattern} search, n next match, N previous match, q quit

Finding names and finding text

command

```
find . -type f -name '*.txt'
```

```
find . -type f -iname '*.TXT'
```

```
grep -n 'ERROR' logs/app.log
```

```
grep -Rni -- 'todo' docs src
```

find

Find paths by properties such as name or type.

grep

Find matching lines inside files.

-n / -i / -R

Line numbers / ignore case / recurse.

Quote the find pattern so find receives *.txt instead of the shell expanding it.

Fewer keystrokes in Bash

Key	Action
Tab / Tab twice	Complete a name / list possible completions
Up / Down	Browse command history
Ctrl+A / Ctrl+E	Beginning / end of the command line
Ctrl+U / Ctrl+K	Cut backward / forward to the line boundary
Ctrl+W / Ctrl+Y	Cut the previous word / paste cut text
Ctrl+C / Ctrl+L	Interrupt / clear the visible screen

History, search, and !!

INTERACTIVE BASH

\$!!

\$ history 10

\$!{command history #}

\$!!:p

!!

Expand and execute the previous command.

!!:p

Print the expansion without executing it.

Ctrl+R

Search history. Enter runs the match.

Repetition repeats side effects too. Practise history expansion with harmless commands.

Aliases for commands you repeat

REMOTE BASH

```
$ alias ll='ls -alh'
```

```
$ alias croot='cd ~/hexploit-linux'
```

```
$ \ls runs the command unaliased
```

```
$ type ll
```

```
$ unalias ll
```

alias

Give a command sequence a short name.

type

See what a command name resolves to.

unalias

Remove an alias from this shell.

Aliases created here last for this shell session. ~/.bashrc can make them persistent.

BREAK

10-minute break

Esc

Next: a text editor that treats keys as commands

VIM

Vim

vim

Edit text directly inside your SSH session

Launch from ~/hexploit-linux: `vim -u vim/workshop.vim vim/practice.txt`

Before one begins mastering VIM, one must learn how to exit VTM.

- Mako

\$ vim

esc

:q



The modes change what keys mean

Mode	Purpose
Normal	Move and edit. Vim starts here.
Insert	Type text. Enter with i, return with Esc.
Visual	Select text. Enter with v or V.
Command-line	Run : commands. Enter submits, Esc cancels.

Saving and leaving Vim

Command	Effect
<code>:w</code>	Write the current buffer to its file
<code>:q</code>	Close the window, if changes permit it
<code>:wq</code>	Write, then close the window
<code>:q!</code>	Close and discard unsaved changes when abandoning the buffer
<code>:w copy.txt</code>	Write the buffer to another file

Press Esc first. Type the colon command. Press Enter.

Entering text exactly where you want

Key	Insert position
i / a	Before / after the cursor
I / A	First nonblank position / end of the line
o / O	New line below / above
r + character	Replace one character, then stay in Normal
Esc	Return to Normal mode

Try A to append to a line, then Esc. Try o to add the next line.

Copy, cut, and put

Keys

Action

yy / 3yy

Copy one / three complete lines

dd / 3dd

Cut one / three complete lines

p / P

Put after / before the cursor

Linewise text goes below or above the current line. Characterwise text goes after or before the cursor.

Undo, redo, and the dot command

Key	What repeats or reverses
u	Undo the last change
Ctrl+R	Redo an undone change
.	Repeat the last change
3.	Repeat a change with a count when supported

Movement by word, line, and screen

Keys	Destination
<code>h j k l</code>	Left, down, up, right
<code>w / b / e</code>	Next word start / previous start / word end
<code>0 / ^ / \$</code>	Line start / first nonblank / line end
<code>gg / G / 12G</code>	First line / last line / line 12
<code>5j / 3w</code>	Five lines down / three words forward
<code>Ctrl+D / Ctrl+U / zz</code>	Half-screen down / up / centre current line

Arrow keys work too. Word and text-object motions are where Vim starts saving effort.

Vim's editing grammar

NORMAL MODE

d + w = dw

c + \$ = c\$

y + \$ = y\$

2 + d + d = 2dd

d

Delete into a register

c

Change, then enter Insert mode

y

Yank: copy into a register

Count

Repeat a motion or operation

dw deletes to the next word start. dd, cc, and yy operate on complete lines.

Text objects describe the target

NORMAL MODE

ciw change inner word

daw delete a word + space

ci" change inside quotes

di(delete inside parentheses

yip yank inner paragraph

i
Inner: leave surrounding delimiters or spacing.

a
Around: include delimiters or surrounding space.

Cursor
Place it in the target word or object.

Search as a movement command

Keys

Action

/text + Enter

Search forward

?text + Enter

Search backward

n / N

Repeat in the search direction / reverse it

* / #

Search the word under the cursor forward / backward

:noh

Clear the current search highlighting

Search and replace with confirmation

VIM COMMAND-LINE

`:{range}s/{old}/{new}/{flags if needed}`

`:%s/old/new/g`

`:2,5s/old/new/gc`

% or 2,5

Whole file or a line range

g

So this on every line

c

Confirm for each replacement

i

Ignore case

At a confirmation prompt: y yes, n no, a all remaining, q stop.

Visual selection

Keys	Selection or edit
v	Character selection
V	Whole-line selection
Motion keys	Extend or shrink the selection
d / y / c	Delete / yank / change the selection
> / <	Indent / unindent selected lines
gv	Reselect the previous selection

v selects characters. V selects lines. Ctrl+V selects a rectangle in the extension.

VIM

Practice Vim

\$ vimtutor



Thanks